第八章作业 参考答案

目录

1 快速排序的分割 **1**
(1) Lomuto 分割 1
(2) 教材算法 8.6 的分割 2

2 MSD 与 LSD 基数排序 **2**
(1) MSD（高位优先） 2
(2) LSD（低位优先） 3
(3) 两种方法的比较与稳定性要求 3

3 Shell 排序与排序算法的性质 **4**
(1) Shell 排序过程 4
(2) 稳定性 4
(3) 基于比较的排序的下界 5

4 子集和的生成与排序 **6**
(1) 增量生成后排序 6
(2) 边生成边合并 6

1 快速排序的分割

对应编程题：力扣 912. 排序数组（快速排序）、力扣 215. 数组中的第 K 个最大元素（快速选择）。

M912. 排序数组

https://leetcode.cn/problems/sort-an-array/

M215. 数组中的第 K 个最大元素

https://leetcode.cn/problems/kth-largest-element-in-an-array/

(1) Lomuto 分割

轴值 `pivot = 40`。`i` 初值为 -1（下标从 0 起），`j` 从 0 扫到 10；`a[j] <= 40` 时令 `i = i + 1` 并交换 `a[i]` 与 `a[j]`。只列出发生实际位置变化的交换：

序号	j 处元素	交换	交换后序列
1	18	18 ↔ 43	18, 43, 65, 27, 52, 9, 34, 71, 26, 60, 15, 40
2	27	27 ↔ 43	18, 27, 65, 43, 52, 9, 34, 71, 26, 60, 15, 40
3	9	9 ↔ 65	18, 27, 9, 43, 52, 65, 34, 71, 26, 60, 15, 40
4	34	34 ↔ 43	18, 27, 9, 34, 52, 65, 43, 71, 26, 60, 15, 40
5	26	26 ↔ 52	18, 27, 9, 34, 26, 65, 43, 71, 52, 60, 15, 40
6	15	15 ↔ 65	18, 27, 9, 34, 26, 15, 43, 71, 52, 60, 65, 40
7	轴值	40 ↔ 43	18, 27, 9, 34, 26, 15, **40**, 71, 52, 60, 65, 43

扫描结束时 `i = 5`，轴值交换到位置 `i + 1 = 6`（下标从 0 起），即第 7 个位置（从 1 起计）。轴值左侧 6 个元素均不大于 40，右侧 5 个元素均大于 40。

(2) 教材算法 8.6 的分割

`left = 0`、`right = 11`，轴值下标为$\lfloor (0+11)/2 \rfloor = 5$，即轴值 **9**。先把它与末尾的 40 交换，并保存到临时变量：

```
43, 18, 65, 27, 52, 40, 34, 71, 26, 60, 15, [9]      l = 0, r = 11，末尾为空闲位置
```

- `l` 从左向右越过不大于 9 的记录：`a[0] = 43 > 9`，立即停下，`l = 0`。因 `l < r`，把 43 放到空闲位置 `r = 11`，`r` 左移到 10，位置 0 空闲。
- `r` 从右向左越过不小于 9 的记录：15、60、26、71、34、40、52、27、65、18 全都不小于 9，`r` 一直减到 0 与 `l` 相遇，循环结束。
- 把轴值回填到空闲位置 0。

分割结束时的序列：

```
[9], 18, 65, 27, 52, 40, 34, 71, 26, 60, 15, 43
```

轴值位于位置 0（第 1 个位置）。左子序列为空，右子序列含其余 11 个记录。

这是最坏情形：轴值 9 恰好是整个序列的最小值，分割没有把序列分成两半，而是只让轴值一个记录就位，下一层递归仍要处理$n-1$ 个记录。若每层都如此，总时间退化为$\Theta(n^2)$。“取中间记录为轴值” 只能保证对正序、逆序输入分得均匀，对本序列这样的随机输入仍可能选到极值；想稳定地避免最坏情形，可以改用 “三者取中” 或随机选取轴值。

2 MSD 与 LSD 基数排序

对应编程题：力扣 164. 最大间距（线性时间排序：桶排序 / 基数排序）。

H164. 最大间距

https://leetcode.cn/problems/maximum-gap/

(1) MSD（高位优先）

第一趟按十位分桶，桶内保持输入时的相对顺序：

桶（十位）	0	1	2	3	4	5	6	7	8	9
记录	[06]	[17, 13]	[25]	[31]	[42a, 42b]	[58]	[65]	[71]	[80]	[93]

第二趟只需递归处理含多个记录的桶：

- 十位为 1 的桶按个位再分：3: [13]、7: [17]。
- 十位为 4 的桶按个位再分：2: [42a, 42b]（同一子桶，保持原顺序）。

其余桶均只有一个记录，不必再分。按桶序依次连接后得到有序序列：

```
06, 13, 17, 25, 31, 42a, 42b, 58, 65, 71, 80, 93
```

(2) LSD（低位优先）

第一趟按个位分桶：

个位桶	0	1	2	3	4	5	6	7	8	9
记录	[80]	[71, 31]	[42a, 42b]	[93, 13]	[]	[25, 65]	[06]	[17]	[58]	[]

第一趟收集结果：

```
80, 71, 31, 42a, 42b, 93, 13, 25, 65, 06, 17, 58
```

第二趟按十位分桶（输入即上面的收集结果）：

十位桶	0	1	2	3	4	5	6	7	8	9
记录	[06]	[13, 17]	[25]	[31]	[42a, 42b]	[58]	[65]	[71]	[80]	[93]

第二趟收集结果（最终有序）：

```
06, 13, 17, 25, 31, 42a, 42b, 58, 65, 71, 80, 93
```

(3) 两种方法的比较与稳定性要求

计算机常用 LSD 的原因。从本题过程可以看到，LSD 每一趟都对全部记录做一次” 分配到 `r` 个桶、按桶序收集”，`d` 位排序码就是完全相同的 `d` 趟，用一个固定的桶（队列）数组循环即可，代码是两层简单循环，时间$O(d(n+r))$。MSD 第一趟之后每个桶都要单独递归处理：本题第二趟要分别进入十位为 1 和 4 的两个桶，一般情况下子桶数随趟数按r^d 增长，需要递归或显式栈来管理各桶的边界，还要把许多很小的桶逐一收集，控制逻辑复杂、常数更大。所以手工排序时 MSD 直观，程序实现时 LSD 更简洁、更快。

稳定性要求。LSD 的正确性本身依赖每一趟分桶是稳定的：第二趟按十位分桶时，13 与 17 之所以在桶内排成 13，17，是因为第一趟已按个位排好而第二趟按先后顺序入桶、按先后顺序出桶（队列）。若某趟不保持先后次序，前面各趟的工作就会被破坏。MSD 的正确性只依赖各位的分桶，不依赖稳定性，但要让 42a 仍排在 42b 之前，每一级分桶（本题是十位分桶和十位为 4 的桶的个位分桶）同样必须按输入先后顺序入桶、出桶。两种方法用队列实现桶时都满足这一要求，因此基数排序是稳定的排序方法。

3 Shell 排序与排序算法的性质

对应编程题：力扣 912. 排序数组、力扣 1122. 数组的相对排序。

E1122. 数组的相对排序

https://leetcode.cn/problems/relative-sort-array/

(1) Shell 排序过程

n = 10，增量依次为 5、2、1。每一趟把下标相差增量的记录看作一个子序列做直接插入排序（直接插入只在前一记录严格大于当前记录时后移，所以同一子序列内相等记录不会互换）。

增量 5。子序列为下标 (0,5)、(1,6)、(2,7)、(3,8)、(4,9)：

子序列	排序前	排序后
(0,5)	36, 23	23, 36
(1,6)	82, 91	82, 91
(2,7)	15, 58	15, 58
(3,8)	67, 36’	36’, 67
(4,9)	49, 7	7, 49

```
增量 5 后: 23, 82, 15, 36', 7, 36, 91, 58, 67, 49
```

增量 2。子序列为偶数下标 23，15，7，91，67 和奇数下标 82，36'，36，58，49，分别插入排序得 7，15，23，67，91 和 36'，36，49，58，82（36 插入时前一记录 36' 不大于它，不移动）：

```
增量 2 后: 7, 36', 15, 36, 23, 49, 67, 58, 91, 82
```

增量 1。对整个序列做直接插入排序：

```
增量 1 后: 7, 15, 23, 36', 36, 49, 58, 67, 82, 91
```

(2) 稳定性

输入中 36 在 36' 之前，而排序结果中 36' 排到了 36 之前，所以 **Shell 排序不稳定**。原因是增量 5 那一趟中 36' 与 67 同在一个子序列、36 与 23 同在另一个子序列，相等的记录被分到不同子序列后各自移动，相对次序无法保证。

教材第 8 章各算法的稳定性：

算法	稳定性	说明或反例（下标表示相同关键字的不同记录）
直接插入排序	稳定	只在前一记录严格大于当前记录时后移，相等记录不越过
Shell 排序	不稳定	本题 36' 越过 36
冒泡排序	稳定	只交换相邻的、严格逆置的记录对
直接选择排序	不稳定	2a，2b，1：第 1 趟把最小记录 1 与 2a 交换，得 1，2b，2a
堆排序	不稳定	2a，2b，1：它已是最大堆，堆顶 2a 与末尾 1 交换得 1，2b，2a，调整得 2b，1，2a，再交换得 1，2b，2a
快速排序	不稳定	2a，2b，1：以 1 为轴值的 Lomuto 分割直接把 1 与 2a 交换，得 1，2b，2a；教材算法 8.6 对 2a，2b，2c 排出 2c，2a，2b
归并排序	稳定	合并时两边关键字相等则先取左边的记录
桶式排序、基数排序	稳定	桶用队列实现，先入先出

(3) 基于比较的排序的下界

判定树。把一个比较排序算法对 n 个互不相同的记录的执行过程画成二叉树：每个内部结点是一次比较$k_i < k_j$，两个分支对应比较结果，每个叶结点对应算法终止时得到的一种排列。算法必须能正确排好任何输入，而 n 个记录共有$n!$ 种可能的初始排列，每种排列必须到达各不相同的叶结点（否则两种不同输入会被同一个" 结论" 处理，至少一个排错），所以判定树至少有$n!$ 个叶结点。

高度为h（根到叶最多h 次比较）的二叉树至多有2^h 个叶结点，因此

$$2^h \geq n! \quad \Longrightarrow \quad h \geq \log_2 n!,$$

而比较次数是整数，所以最坏情况下至少$\lceil \log_2 n! \rceil$ 次比较。又

$$\log_2 n! = \sum_{i=1}^{n} \log_2 i \;\geq\; \sum_{i=\lceil n/2 \rceil}^{n} \log_2 \frac{n}{2} \;\geq\; \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n),$$

故任何基于比较的排序算法最坏情况时间为$\Omega(n \log n)$。

n = 5。$5! = 120$, $\log_2 120 \approx 6.91$, 下界为$\lceil 6.91 \rceil = 7$ 次比较。教材习题 1 要求 "最多只用 7 次比较" 排好 5 个元素, 说明这个下界对$n = 5$ 是可以达到的 (归并插入排序, 即 Ford–Johnson 算法，恰好用 7 次)；而$n = 5$ 时直接插入排序最坏需要 10 次，堆排序、归并排序也多于 7 次。

4 子集和的生成与排序

对应编程题：力扣 78. 子集（枚举全部子集）。

M78. 子集

https://leetcode.cn/problems/subsets/

(1) 增量生成后排序

用增量法生成：初始只有空集的和 0；每读入一个元素 x，把当前每个已有的和 s 对应的新和 s + x 追加到数组末尾（相当于子集" 不含 x"与" 含 x"两种情形）。最后对全部2^n 个和排序。

```
AllSubsetSumsSorted(A):
    sums = [0]
    for each x in A:
        m = length(sums)
        for i = 0 to m - 1:
            append sums[i] + x to sums
    sort sums in nondecreasing order     // 堆排序或归并排序
    return sums
```

对$A = \{3, 7, 11, 14\}$，生成阶段依次得到

```
[0]
[0, 3]
[0, 3, 7, 10]
[0, 3, 7, 10, 11, 14, 18, 21]
[0, 3, 7, 10, 11, 14, 18, 21, 14, 17, 21, 24, 25, 28, 32, 35]
```

排序后的完整输出（16 个）：

```
0, 3, 7, 10, 11, 14, 14, 17, 18, 21, 21, 24, 25, 28, 32, 35
```

两个相同的和（14 既是单元素子集 {14} 的和也是 3 + 11，21 既是 7 + 14 也是 3 + 7 + 11）来自不同子集，都应保留。

时间复杂度。生成阶段共做$1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1$ 次加法，用$O(2^n)$ 空间；用比较排序对$N = 2^n$ 个数排序需

$$O(N \log N) = O(2^n \log 2^n) = O(n\, 2^n)$$

时间，排序是主要开销，总时间为$O(n\, 2^n)$。

(2) 边生成边合并

设处理完前 k 个元素后的子集和序列 S 已经非降有序。加入元素 x 时，序列 S + x（S 中每个数加 x）只是把 S 整体平移，因此同样非降有序；新的子集和集合恰好是 S 与 S + x 的并（多重集），而两个有序序列可以用归并排序中的 Merge 过程在线性时间内合并成一个有序序列。于是每一步的结果都保持有序，无需最后再排序：

```
AllSubsetSumsByMerge(A):
    S = [0]
```

```
    for each x in A:
        T = [s + x for s in S]        // 仍有序
        S = Merge(S, T)               // 两个有序序列归并
    return S
```

对本题集合：

```
[0]
Merge([0], [3])                          -> [0, 3]
Merge([0, 3], [7, 10])                   -> [0, 3, 7, 10]
Merge([0, 3, 7, 10], [11, 14, 18, 21])  -> [0, 3, 7, 10, 11, 14, 18, 21]
Merge([0, 3, 7, 10, 11, 14, 18, 21], [14, 17, 21, 24, 25, 28, 32, 35])
    -> [0, 3, 7, 10, 11, 14, 14, 17, 18, 21, 21, 24, 25, 28, 32, 35]
```

第 k 步合并两个长度为2^k 的序列，代价$O(2^k)$；总时间

$$\sum_{k=0}^{n-1} O(2^k) = O(2^n),$$

空间同为$O(2^n)$。与第 (1) 问相比省去了$\log 2^n = n$ 的因子，达到输出规模的线性时间，已是最优（至少要写出2^n 个数）。

> 核对说明：第 1 题第 (1) 问、第 2 题第 (1)(2) 问、第 4 题第 (1) 问取自本次作业的原始题目，原答案经脚本逐题复核无误。新增内容为：第 1 题第 (2) 问（教材算法 8.6 的分割）、第 2 题第 (3) 问、第 3 题（依据教材 8.2.2、8.7.3 节）以及第 4 题第 (2) 问。